

Modelos de Deep Learning

Introducción a Transformers

Universidad ORT Uruguay

17 de Noviembre, 2025

De la Atención a los Transformers

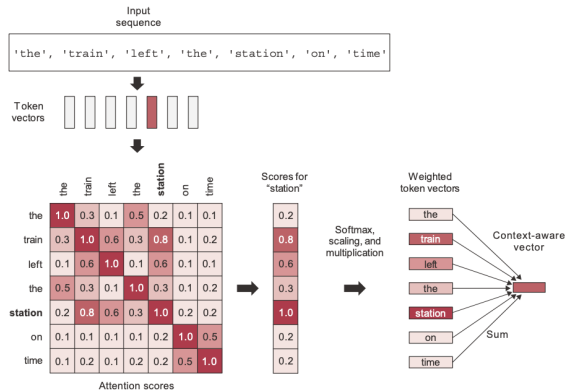
- [Vaswani et al., 2017] publican "*Attention Is All You Need*".
- Descubren que la atención por sí sola puede reemplazar a las RNNs.
- El **Transformer** utiliza únicamente mecanismos de atención para:
 - Modelar dependencias a largo plazo.
 - Procesar secuencias en paralelo (sin recurrencia).
 - Escalar a grandes volúmenes de datos.
- Resultado: revolución en NLP y posteriormente en visión, audio y más.

Motivación del Self-Attention

- **Word embeddings** tradicionales asignan un solo vector fijo a cada palabra.
- Pero el significado real depende del contexto:
 - “*go on a date*” $\neq$ “*mark the date*”
 - “*see you soon*” $\neq$ “*see what you mean*”
- Necesitamos representaciones que sean **sensibles al contexto**.
- **Self-attention** ajusta cada palabra según las demás del mismo enunciado.

Cómo funciona el Self-Attention

1. Para cada palabra, se calculan **scores de relevancia** con todas las demás (dot product).
2. Los scores indican qué tan relacionadas están las palabras.
3. Se aplica **softmax** para normalizar los pesos.
4. Se realiza una suma ponderada de todos los vectores → nueva representación contextual.

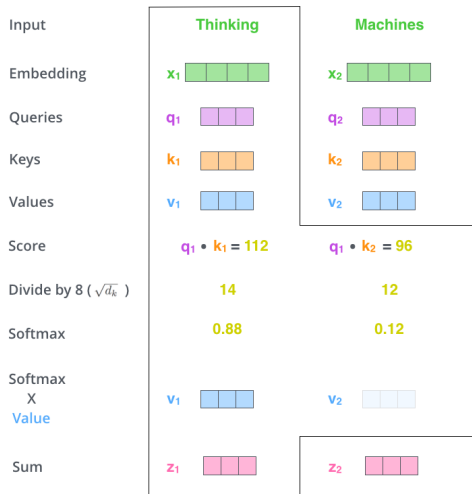


Ejemplo: "The train left the station on time" → el nuevo vector de "station" incorpora el contexto de "train".

Intuición del Self-Attention

- Cada palabra “mira” al resto del contexto para redefinir su significado.
- La nueva representación (*context-aware vector*) combina:
 - Información de la palabra original.
 - Información de palabras relevantes (con mayor peso).
- Resultado: **representaciones dinámicas y contextuales.**

Ejemplo: Self-Attention



- Cada palabra se transforma en tres vectores: **Query (q)**, **Key (k)** y **Value (v)**.
- Se calcula la similitud entre q_i y todos los k_j (producto punto) $\rightarrow$ **attention scores**.
- Los scores se escalan y normalizan con **Softmax** para obtener pesos.
- Cada palabra produce una nueva representación (z_i) como suma ponderada de los valores v_j .
- Resultado: un vector contextual que depende de las demás palabras de la secuencia.

Dot-Product Attention

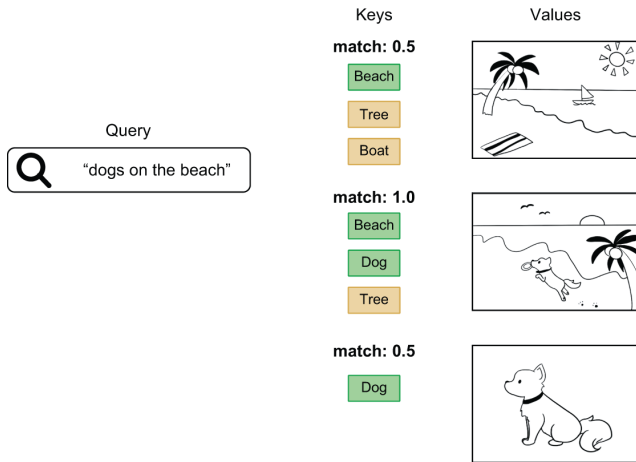
- Objetivo: determinar qué partes del **input** (secuencia fuente) son más relevantes para generar cada palabra del **output**.
- Para cada palabra destino, se calcula una puntuación de similitud con cada palabra fuente.
- Se obtiene una **distribución de atención** (softmax) → pesos que suman 1.
- El vector de salida es una **suma ponderada** de los vectores fuente.



Ejemplo: "te traeré la bolsa" → el modelo asigna más peso a "I", "bring" al traducir "traeré".

Queries, Keys y Values

Analogía: buscar “dogs on the beach” → los **keys** son tags, los **values** las imágenes.



Multi-Head Attention

- Un solo “head” puede captar solo un tipo de relación (ej. sujeto-verbo).
- Los **Transformers** usan varias cabezas de atención en paralelo:
- Cada una aprende distintas relaciones semánticas.
- Se concatenan y proyectan juntas → representación más rica.



Cada cabeza de atención observa la secuencia desde una perspectiva diferente.

Bloque Encoder del Transformer

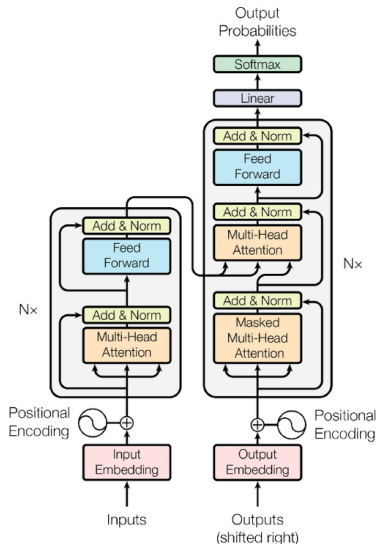
- Cada token atiende a todos los demás de la secuencia (incluido a sí mismo).
- Estructura interna:
 1. Multi-Head Self-Attention (+ máscara de padding)
 2. Suma residual + Layer Normalization
 3. Feed-Forward Network (2 capas densas + activación)
 4. Suma residual + Layer Normalization
- Salida tiene misma dimensión que entrada $\rightarrow$ se pueden apilar N encoders.

Bloque Decoder del Transformer

- Similar al encoder, pero incorpora dos tipos de atención:
 1. **Masked Self-Attention:** cada posición sólo puede atender a tokens previos (causal mask).
 2. **Cross-Attention:** combina la salida del encoder con el estado del decoder.
 3. Feed-Forward + Residual + LayerNorm.
- El decoder genera la secuencia de salida palabra por palabra.
- También se apilan N decoders, compartiendo la misma estructura.

Componentes Clave del Transformer

- **Multi-Head Attention:** múltiples “cabezas” aprenden relaciones distintas en paralelo.
- **Feed-Forward Network:** introduce no linealidad y actualiza cada token individualmente.
- **Residual Connections:** facilitan el flujo de gradientes en redes profundas.
- **Layer Normalization:** normaliza cada secuencia de forma independiente.
- **Positional Encoding:** agrega información de orden, ya que el modelo no es secuencial.



Bibliografía

 Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017).

Attention is all you need.

In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.